Mini Project: Parking Management System

Submitted by:

Rupesh Singh USN ID: 25MCAR0017

Vishwajeet Singh USN ID: 25MCAR0219

Code:

```
from collections import deque
from dataclasses import dataclass
from datetime import datetime
import math

# Billing Rates (modifiable)
CAR_RATE = 20      # ₹20 per hour
BIKE_RATE = 10     # ₹10 per hour


@dataclass
class Vehicle:
    reg_no: str
    vehicle_type: str  # 'car' or 'bike'
    owner: str = ""
    entry_time: datetime = None  # datetime object

    def to_dict(self):
        return {
            "reg_no": self.reg_no,
            "vehicle_type": self.vehicle_type,
            "owner": self.owner,
```

```
            "entry_time": self.entry_time.strftime("%Y-%m-%d
%H:%M:%S") if self.entry_time else None
        }


class ParkingLot:
    def __init__(self, car_capacity=5, bike_capacity=5):
        self.car_slots = [None] * car_capacity
        self.bike_slots = [None] * bike_capacity
        self.lookup = {}  # reg_no -> (type, index)
        self.waitlist = {"car": deque(), "bike": deque()}

    # ---- Helper utilities ----
    def _now(self):
        return datetime.now()

    def available_slots(self):
        """Return available counts for car and bike."""
        return {
            "car": sum(1 for s in self.car_slots if s is None),
            "bike": sum(1 for s in self.bike_slots if s is None),
        }

    def slot_status_list(self, vehicle_type):
        """Return list of booleans for each slot: True if occupied, False if
vacant."""
        slots = self.car_slots if vehicle_type == "car" else self.bike_slots
        return [s is not None for s in slots]

    def pretty_slot_status(self, vehicle_type):
        """Return a human readable list of 'Slot X: Vacant/Filled' strings
for a type."""
        status = self.slot_status_list(vehicle_type)
```

```
        lines = []
        for i, occupied in enumerate(status, start=1):
            lines.append(f"Slot {i}: {'Filled' if occupied else 'Vacant'}")
        return lines

    def _find_first_free_slot(self, vtype):
        slots = self.car_slots if vtype == "car" else self.bike_slots
        for i, s in enumerate(slots):
            if s is None:
                return i
        return None

    # --- Park vehicle ---
    def park_vehicle(self, reg_no, vtype, owner=""):
        vtype = vtype.lower()
        if vtype not in ("car", "bike"):
            return False, "Invalid vehicle type. Use 'car' or 'bike'."

        if reg_no in self.lookup:
            return False, "This registration is already parked."

        # snapshot of slot status before attempt (for message)
        before_status = self.pretty_slot_status(vtype)
        free_slot = self._find_first_free__slot(vtype)
        vehicle = Vehicle(reg_no=reg_no, vehicle_type=vtype,
owner=owner, entry_time=self._now())

        if free_slot is not None:
            # was Vacant -> will be Filled
            if vtype == "car":
                self.car_slots[free_slot] = vehicle
            else:
                self.bike_slots[free_slot] = vehicle
```

```
            self.lookup[reg_no] = (vtype, free_slot)
            # status after
            after_status = self.pretty_slot_status(vtype)
            avail = self.available_slots()
            msg_lines = [
                f"Vehicle parked successfully.",
                f"Type      : {vtype}",
                f"Slot No   : {free_slot + 1}",
                f"Entry Time: {vehicle.entry_time.strftime('%Y-%m-%d
%H:%M:%S')}",
                "",
                "Slot status (before -> after):"
            ]
            # produce line-by-line before -> after for the particular slot
only (more compact)
            before_line = f"Slot {free_slot+1}: {'Filled' if 'Filled' in
before_status[free_slot] else 'Vacant'}"
            after_line = f"Slot {free_slot+1}: {'Filled' if 'Filled' in
after_status[free_slot] else 'Vacant'}"
            msg_lines.append(f"{before_line} -> {after_line}")
            msg_lines.append("")
            msg_lines.append(f"Available - Cars: {avail['car']}, Bikes:
{avail['bike']}")
            return True, "\n".join(msg_lines)
        else:
            # no free slot -> add to waitlist
            self.waitlist[vtype].append(vehicle)
            avail = self.available_slots()
            msg_lines = [
                f"No available {vtype} slots. Added to {vtype} waitlist at
position {len(self.waitlist[vtype])}.",
                "",
```

```
            f"All {vtype} slots are currently Filled.",
            f"Available - Cars: {avail['car']}, Bikes: {avail['bike']}"
        ]
        return False, "\n".join(msg_lines)

    # --- Leave vehicle + billing ---
    def leave_vehicle(self, reg_no):
        if reg_no not in self.lookup:
            return False, "Vehicle not found in parking."

        vtype, idx = self.lookup.pop(reg_no)

        # retrieve and clear slot
        slots = self.car_slots if vtype == "car" else self.bike_slots
        vehicle = slots[idx]
        slots[idx] = None

        exit_time = self._now()
        # compute duration in seconds/hours
        duration_seconds = (exit_time -
vehicle.entry_time).total_seconds()
        hours = math.ceil(duration_seconds / 3600) if duration_seconds
> 0 else 1

        rate = CAR_RATE if vtype == "car" else BIKE_RATE
        total = hours * rate

        bill_lines = [
            "--- BILL ---",
            f"Vehicle No : {vehicle.reg_no}",
            f"Type       : {vehicle.vehicle_type}",
            f"Entry Time : {vehicle.entry_time.strftime('%Y-%m-%d
%H:%M:%S')}",
```

```
            f"Exit Time  : {exit_time.strftime('%Y-%m-%d %H:%M:%S')}",
            f"Hours Stayed: {hours} hour(s)",
            f"Rate (/hr) : ₹{rate}",
            f"Total Bill : ₹{total}",
            "--------------------------------"
        ]

        # If waitlist exists for this type, auto-assign next vehicle to freed
slot
        assigned_msg = ""
        if self.waitlist[vtype]:
            next_vehicle = self.waitlist[vtype].popleft()
            next_vehicle.entry_time = self._now()
            if vtype == "car":
                self.car_slots[idx] = next_vehicle
            else:
                self.bike_slots[idx] = next_vehicle
            self.lookup[next_vehicle.reg_no] = (vtype, idx)
            assigned_msg = (f"\nNote: Slot {idx+1} was assigned to
waitlisted vehicle {next_vehicle.reg_no}."
                            f"\nAssigned Entry Time:
{next_vehicle.entry_time.strftime('%Y-%m-%d %H:%M:%S')}")

        # also include updated availability after leave/assignment
        avail = self.available_slots()
        assigned_msg += f"\nAvailable - Cars: {avail['car']}, Bikes:
{avail['bike']}"

        return True, "\n".join(bill_lines) + assigned_msg

    # --- Utility: show status ---
    def show_full_status(self):
        """Return a multi-line status string showing all slots and
```

```
waitlists."""
        lines = []
        lines.append("---- Car Slots ----")
        for i, s in enumerate(self.car_slots, start=1):
            if s:
                lines.append(f"{i}. Filled - {s.reg_no} (Entry:
{s.entry_time.strftime('%Y-%m-%d %H:%M:%S')})")
            else:
                lines.append(f"{i}. Vacant")
        lines.append("\n---- Bike Slots ----")
        for i, s in enumerate(self.bike_slots, start=1):
            if s:
                lines.append(f"{i}. Filled - {s.reg_no} (Entry:
{s.entry_time.strftime('%Y-%m-%d %H:%M:%S')})")
            else:
                lines.append(f"{i}. Vacant")
        lines.append("\n---- Waitlists ----")
        lines.append("Cars waitlist: " + ", ".join([v.reg_no for v in
self.waitlist["car"]]) if self.waitlist["car"] else "Cars waitlist: (empty)")
        lines.append("Bikes waitlist: " + ", ".join([v.reg_no for v in
self.waitlist["bike"]]) if self.waitlist["bike"] else "Bikes waitlist:
(empty)")
        lines.append("\nAvailable counts: Cars: {}, Bikes:
{}".format(*[self.available_slots()[k] for k in ("car", "bike")]))
        return "\n".join(lines)


# --- CLI Demo ---
def print_menu():
    print("\n------ PARKING SYSTEM ------")
    print("1. Park Vehicle")
    print("2. Leave Vehicle")
    print("3. Show Full Status")
```

```
    print("4. Exit")
    print("----------------------------")


def main():
    lot = ParkingLot(car_capacity=5, bike_capacity=5)

    while True:
        print_menu()
        ch = input("Enter choice: ").strip()

        if ch == "1":
            reg = input("Enter Registration No: ").strip()
            vtype = input("Enter Type (car/bike): ").strip()
            owner = input("Owner Name (optional): ").strip()
            ok, msg = lot.park_vehicle(reg, vtype, owner)
            print("\n" + msg)

        elif ch == "2":
            reg = input("Enter Registration No to Exit: ").strip()
            ok, msg = lot.leave_vehicle(reg)
            print("\n" + msg)

        elif ch == "3":
            print("\n" + lot.show_full_status())

        elif ch == "4":
            print("Exiting system...")
            break

        else:
            print("Invalid choice. Try again.")
```

```
if __name__ == "__main__":
    main()
```

Output:

```
----- PARKING SYSTEM ------
1. Park Vehicle
2. Leave Vehicle
3. Show Full Status
4. Exit

Enter choice: 3

---- Car Slots ----
1. Vacant
2. Vacant
3. Vacant
4. Vacant
5. Vacant

---- Bike Slots ----
1. Vacant
2. Vacant
3. Vacant
4. Vacant
```

5. Vacant

---- Waitlists ----

Cars waitlist: (empty)

Bikes waitlist: (empty)

Available counts: Cars: 5, Bikes: 5

------ PARKING SYSTEM ------

1. Park Vehicle

2. Leave Vehicle

3. Show Full Status

4. Exit

Enter choice: 1

Enter Registration No: KA05CL2345

Enter Type (car/bike): car

Owner Name (optional): Rupesh

Vehicle parked successfully.

Type   : car

Slot No   : 1

Entry Time: 2025-12-10 21:27:54

Slot status (before -> after):

Slot 1: Vacant -> Slot 1: Filled

Available - Cars: 4, Bikes: 5

------ PARKING SYSTEM ------

1. Park Vehicle

2. Leave Vehicle

3. Show Full Status

4. Exit

Enter choice: 1

Enter Registration No: KA05CL06345

Enter Type (car/bike): car

Owner Name (optional): 45

Vehicle parked successfully.

Type   : car

Slot No   : 2

Entry Time: 2025-12-10 21:28:22

Slot status (before -> after):

Slot 2: Vacant -> Slot 2: Filled

Available - Cars: 3, Bikes: 5

------ PARKING SYSTEM ------

1. Park Vehicle

2. Leave Vehicle

3. Show Full Status

4. Exit

Enter choice: 1

Enter Registration No: KA05CL0345

Enter Type (car/bike): caar

Owner Name (optional): exzcf

Invalid vehicle type. Use 'car' or 'bike'.

------ PARKING SYSTEM ------

1. Park Vehicle

2. Leave Vehicle

3. Show Full Status

4. Exit

Enter choice: 1

Enter Registration No: KA05KK4560

Enter Type (car/bike): car

Owner Name (optional): 4545

Vehicle parked successfully.

Type   : car

Slot No   : 3

Entry Time: 2025-12-10 21:28:51

Slot status (before -> after):

Slot 3: Vacant -> Slot 3: Filled

Available - Cars: 2, Bikes: 5

------ PARKING SYSTEM ------

1. Park Vehicle

2. Leave Vehicle

3. Show Full Status

4. Exit

Enter choice: 1

Enter Registration No: KA05LR3698

Enter Type (car/bike): car

Owner Name (optional): 555

Vehicle parked successfully.

Type   : car

Slot No   : 4

Entry Time: 2025-12-10 21:29:07

Slot status (before -> after):

Slot 4: Vacant -> Slot 4: Filled

Available - Cars: 1, Bikes: 5

------ PARKING SYSTEM ------

1. Park Vehicle

2. Leave Vehicle

3. Show Full Status

4. Exit

Enter choice: 1

Enter Registration No: KA01CK7896

Enter Type (car/bike): car

Owner Name (optional): 12

Vehicle parked successfully.

Type   : car

Slot No   : 5

Entry Time: 2025-12-10 21:29:20

Slot status (before -> after):

Slot 5: Vacant -> Slot 5: Filled

Available - Cars: 0, Bikes: 5

------ PARKING SYSTEM ------

1. Park Vehicle

2. Leave Vehicle

3. Show Full Status

4. Exit

Enter choice: 1

Enter Registration No: KA02CQ4545

Enter Type (car/bike): car

Owner Name (optional): 444

No available car slots. Added to car waitlist at position 1.

All car slots are currently Filled.

Available - Cars: 0, Bikes: 5

------ PARKING SYSTEM ------

1. Park Vehicle

2. Leave Vehicle

3. Show Full Status

4. Exit

Enter choice: 3

---- Car Slots ----

1. Filled – KA05CL0345 (Entry: 2025-12-10 21:27:54)

2. Filled – KA05KK4560 (Entry: 2025-12-10 21:30:14)

3. Filled – KA05LR3698 (Entry: 2025-12-10 21:28:51)

4. Filled – KA01CK7896 (Entry: 2025-12-10 21:29:07)

5. Filled – KA02CQ4545 (Entry: 2025-12-10 21:29:20)

---- Bike Slots ----

1. Vacant

2. Vacant

3. Vacant

4. Vacant

5. Vacant

---- Waitlists ----

Cars waitlist: hjj45466

Bikes waitlist: (empty)

Available counts: Cars: 0, Bikes: 5

------ PARKING SYSTEM ------

1. Park Vehicle

2. Leave Vehicle

3. Show Full Status

4. Exit

Enter choice: 2

Enter Registration No to Exit: 454

---------- BILL ----------

Vehicle No : KA05KK4560

Type : car

Entry Time : 2025-12-10 21:28:22

Exit Time : 2025-12-10 21:30:14

Hours Stayed: 1 hour(s)

Rate (/hr) : ₹20

Total Bill : ₹20

Note: Slot 2 was assigned to waitlisted vehicle hjj45466.

Assigned Entry Time: 2025-12-10 21:30:14

Available - Cars: 0, Bikes: 5

------ PARKING SYSTEM ------

1. Park Vehicle

2. Leave Vehicle

3. Show Full Status

4. Exit

Enter choice: 3

---- Car Slots ----

1. Filled – KA05CL0345 (Entry: 2025-12-10 21:27:54)

2. Filled – KA05KK4563 (Entry: 2025-12-10 21:30:14)

3. Filled – KA05LR3698 (Entry: 2025-12-10 21:28:51)

4. Filled – KA01CK7896 (Entry: 2025-12-10 21:29:07)

5. Filled – KA02CQ4545 (Entry: 2025-12-10 21:29:20)

---- Bike Slots ----

1. Vacant

2. Vacant

3. Vacant

4. Vacant

5. Vacant

---- Waitlists ----

Cars waitlist: (empty)

Bikes waitlist: (empty)

Available counts: Cars: 0, Bikes: 5

------ PARKING SYSTEM ------

1. Park Vehicle

2. Leave Vehicle

3. Show Full Status

4. Exit

Enter choice: